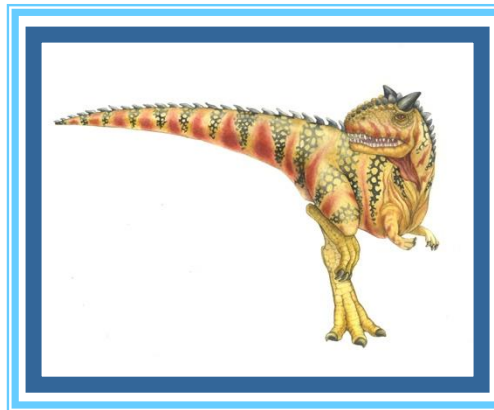


Procesos





Procesos

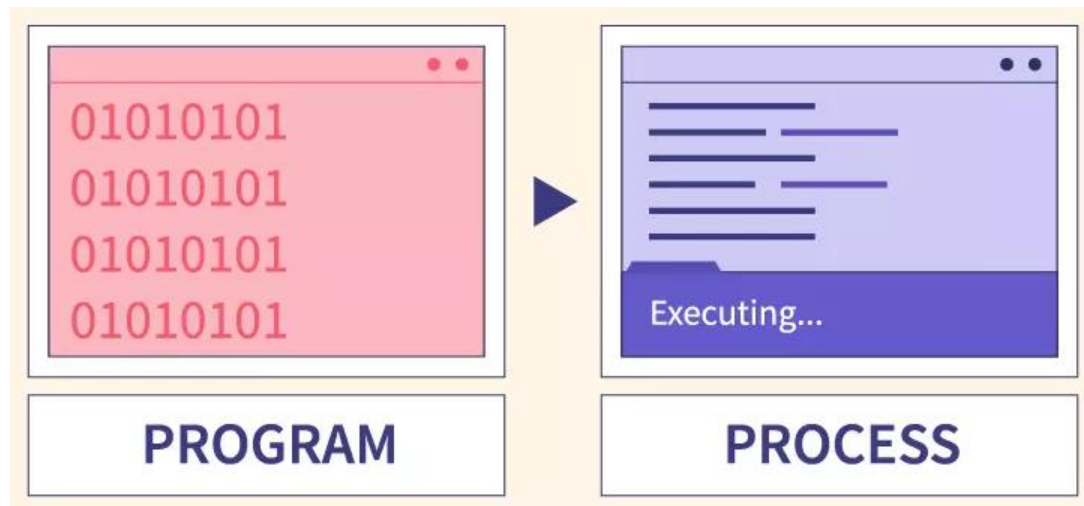
- ❑ ¿Qué es un proceso, como podemos conceptualizarlo?
- ❑ ¿Cuál es el ciclo de vida de un proceso? ¿Qué etapas podría tener?
- ❑ ¿Cómo organiza el sistema operativo los procesos?





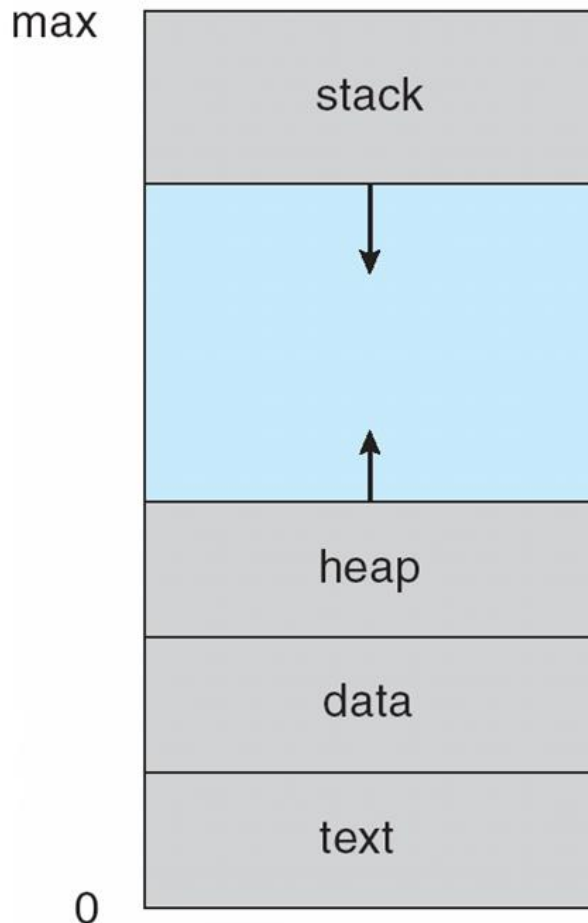
Concepto de proceso

- ❑ **Proceso (secuencial):** programa en ejecución.
 - ❑ Incluye código, *program counter*, registros y variables.
 - ❑ Un proceso sin programa no hace nada, al igual que un programa sin proceso.
- ❑ El programa es pasivo, el proceso es activo.
- ❑ El proceso es la unidad de trabajo para el sistema operativo.
- ❑ El proceso no es dependiente del tiempo.





Concepto de proceso



- Conceptualmente, cada proceso tiene su propio CPU por acarrearlo en un contexto aislado.
 - Dos procesos separados pueden asociarse con un mismo programa.



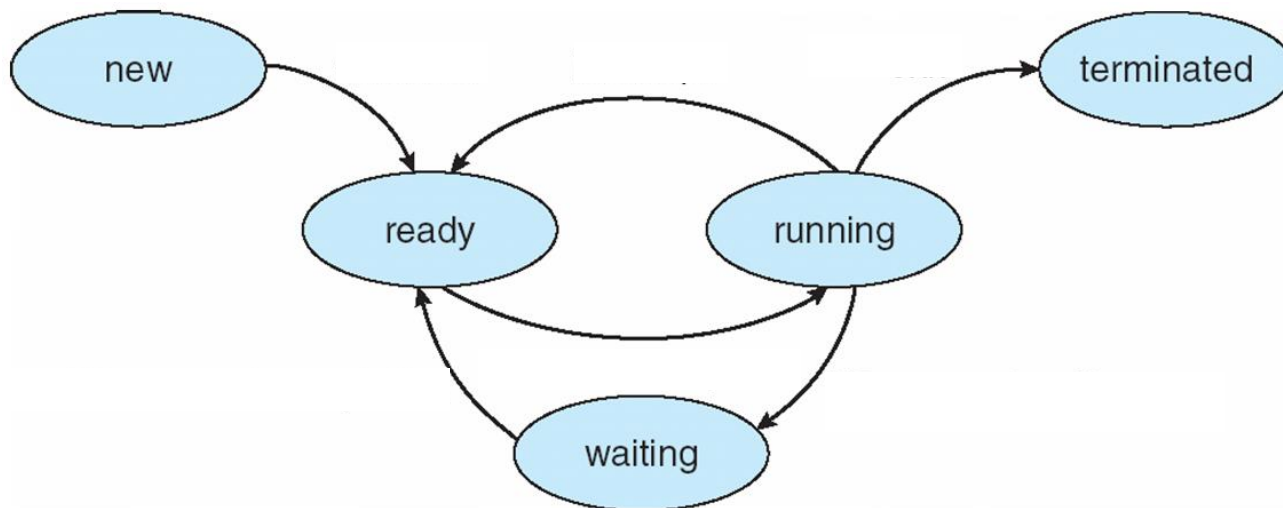
- **Pila**: almacenamiento temporal para operaciones administrativas (parámetros, variables locales, direcciones de retorno...)
- **Data**: representación del proceso.
- **Text**: código siendo ejecutado.
- **Heap**: espacio de memoria asignada de forma dinámica, modificable por el usuario.





Estados de un proceso

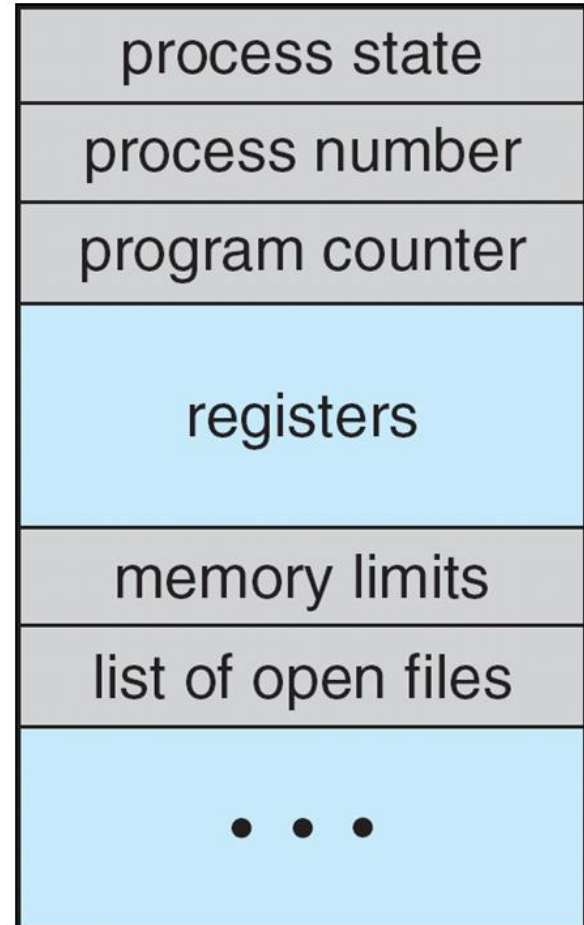
- El **estado** de un proceso cambia a lo largo de su ciclo de vida. Permite tomar decisiones sobre el manejo y la calendarización del proceso.
 - **Nuevo**: proceso creado o en creación, todavía no cargado a memoria.
 - **En ejecución**: el proceso tiene procesador asignado y está corriendo.
 - **Esperando/bloqueado**: incapaz de continuar ejecutándose hasta que ocurra algún evento externo.
 - **Listo**: el proceso está en cola, esperando ejecución.
 - **Terminado**: proceso concluido, listo para ser eliminado.





Process Control Block (PCB)

- El **PCB** es una estructura de datos que representa al proceso en el sistema operativo.
 - Se referencia mediante una **tabla de procesos**.
- Contiene, entre otros:
 - Identificación del proceso.
 - *Program counter*.
 - Contenido de los registros en el CPU siendo usados por el proceso.
 - Información de calendarización de CPU.
 - ▶ Estado del proceso.
 - ▶ Prioridad.
 - Información de manejo de memoria.
 - Información de I/O, como solicitudes pendientes y dispositivos asignados al proceso.





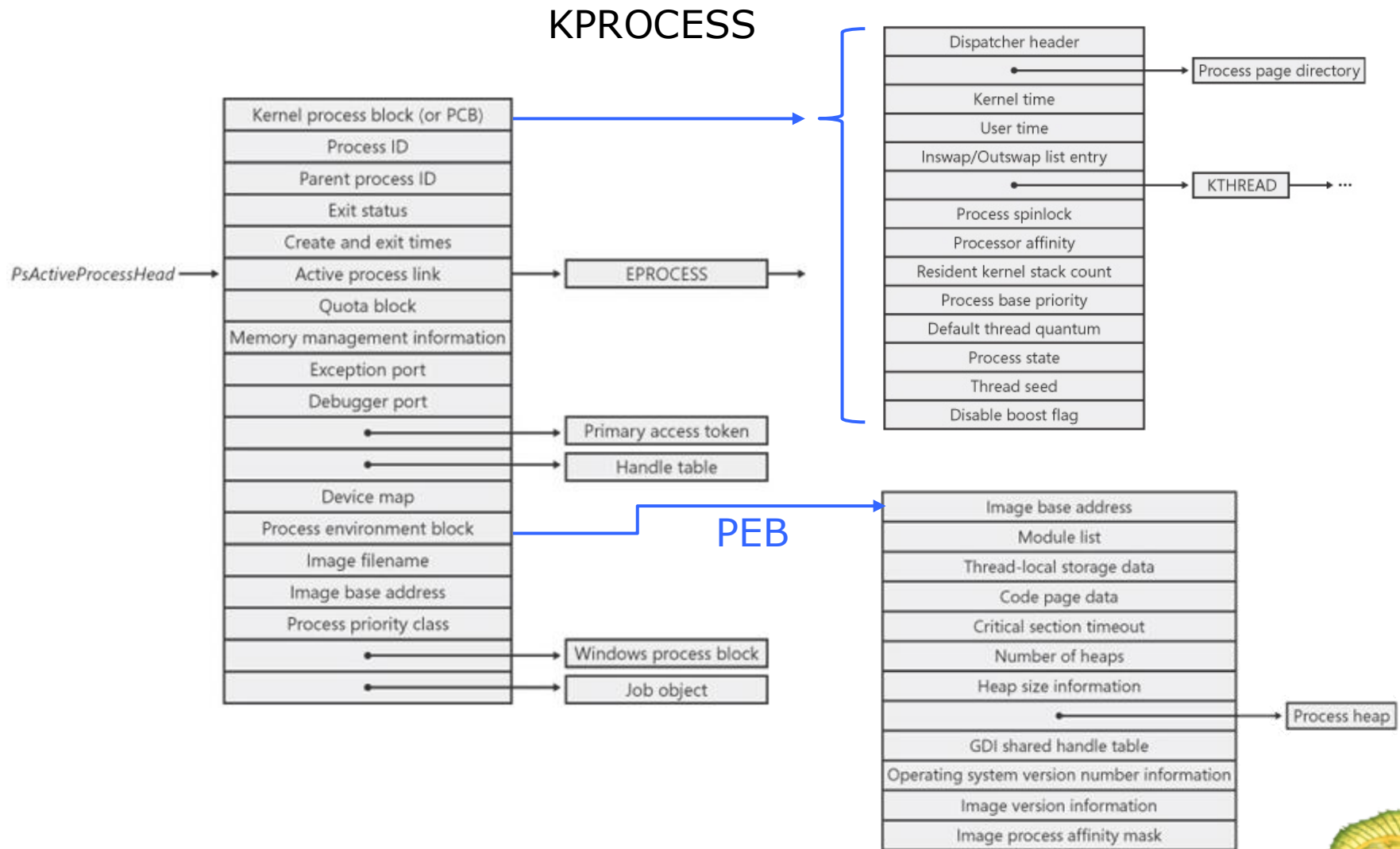
Implementaciones de PCB

Kernel	Process management	File management
Registers	Pointer to text segment	UMASK mask
Program counter	Pointer to data segment	Root directory
Program status word	Pointer to bss segment	Working directory
Stack pointer	Exit status	File descriptors
Process state	Signal status	Real id
Current scheduling priority	Process ID	Effective UID
Maximum scheduling priority	Parent process	Real GID
Scheduling ticks left	Process group	Effective GID
Quantum size	Children's CPU time	Controlling tty
CPU time used	Real UID	Save area for read/write
Message queue pointers	Effective UID	System call parameters
Pending signal bits	Real GID	Various flag bits
Various flag bits	Effective GID	
Process name	File info for sharing text	
	Bitmaps for signals	
	Various flag bits	
	Process name	



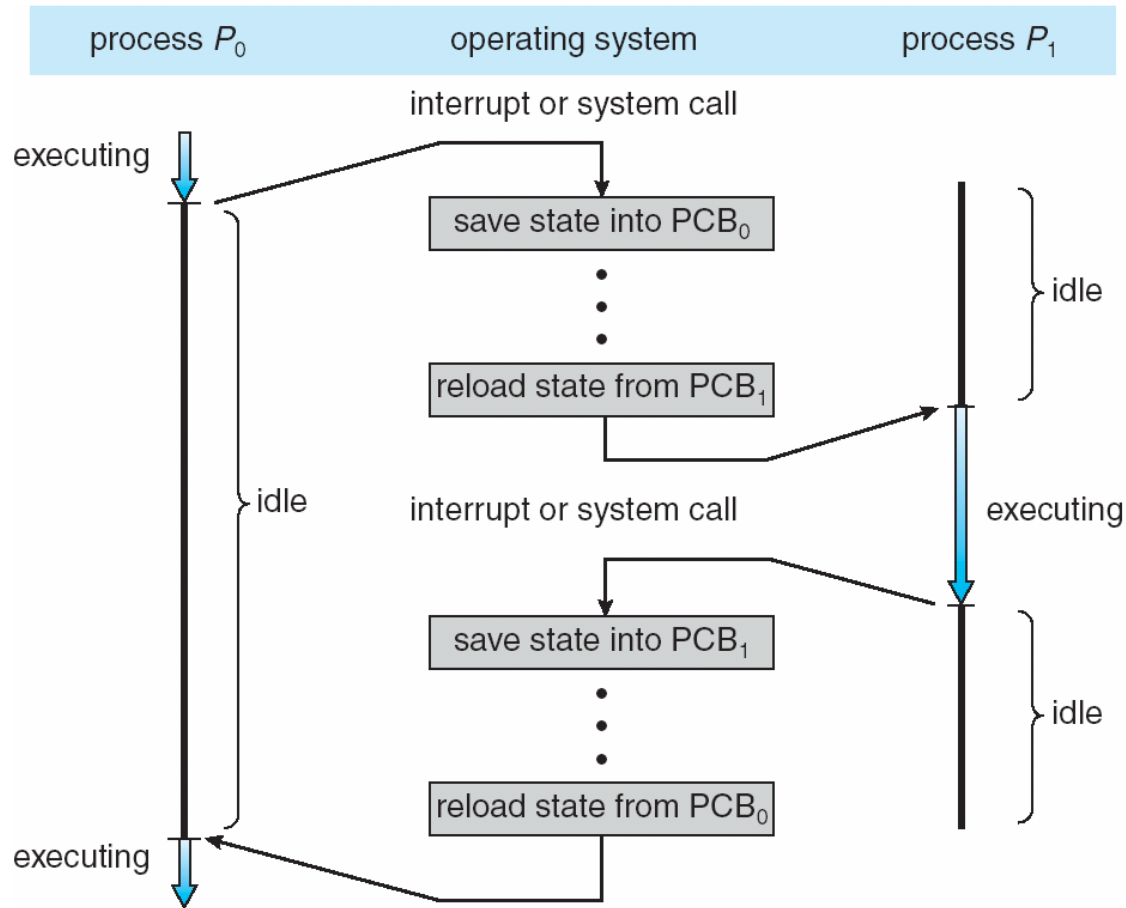


Implementaciones de PCB





Uso de PCB's en *process switching*





Cambio de contexto

- ❑ ¿Qué es un cambio de contexto y, principalmente, qué involucra?
- ❑ ¿Podríamos calificar a los cambios de contexto como buenos o malos, en algún sentido? ¿Cómo y por qué?
- ❑ Ejemplos de cuándo ocurren cambios de contexto.





Cambio de contexto

- El **cambio de contexto** conlleva pausar un proceso, almacenar su información y ejecutar otro proceso.
 - Es tiempo que el procesador invierte en tareas administrativas. Se considera **overhead**.
- El contexto incluye, fundamentalmente, el *program counter* y el valor de los registros usados por el proceso.
 - Regularmente incluye *stack pointers* para preservar valores temporales del proceso.
- Los cambios de contexto pueden ser apoyados por el *hardware*.
 - Ejemplo: el procesador UltraSPARC de Sun Microsystems ofrece varios conjuntos de registros.





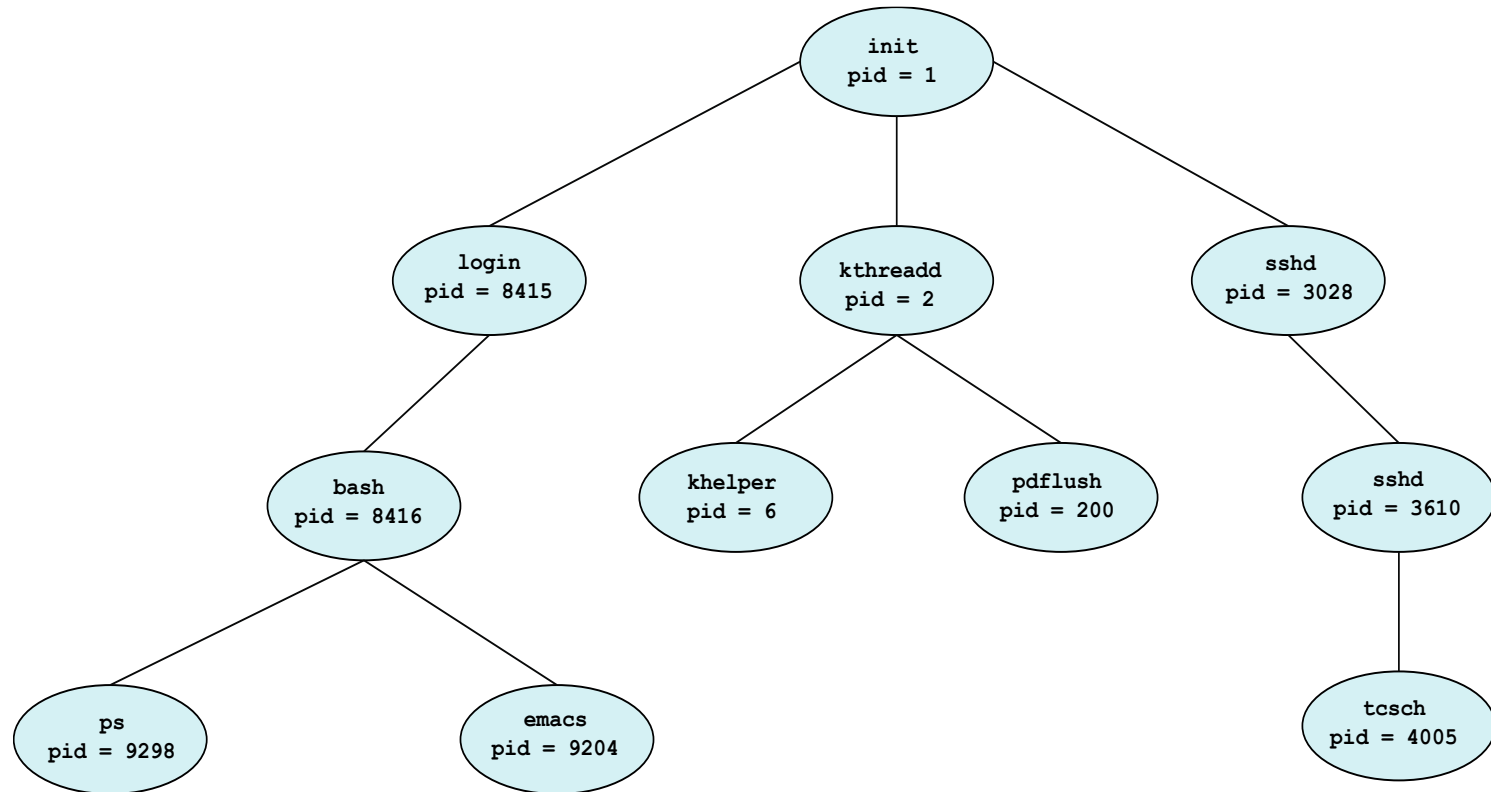
Creación de procesos

- La creación de procesos es manejada por el sistema operativo.
 - El OS crea procesos durante su inicialización, al proveer ciertos servicios y ante solicitudes del usuario.
- Un proceso puede engendrar a otro, estableciendo una **relación padre-hijo**.
 - La relación entre procesos afecta cómo éstos se comunican, aparte de permitir navegar un **árbol de procesos**.
 - En sistemas Linux, los procesos de usuario pertenecen a un mismo árbol cuya raíz es el proceso **init** (o **systemd**).
- Crear un proceso involucra asignarle recursos. Cómo y cuáles recursos, depende de cómo sea creado el proceso.
 - Cuando un proceso engendra a otro puede compartirle ciertos recursos.
 - En sistemas UNIX, los procesos son creados con `fork()` y reciben el mismo programa (*i.e.*, código) y los mismos datos que el proceso padre.
 - ▶ Sistemas Linux también ofrecen la llamada `clone()`, que permite especificar qué recursos son compartidos con el hijo.





Árbol de procesos ejemplar en Linux





Init

```
LXTerminal
File Edit Tabs Help
top - 05:54:35 up 2:43, 3 users, load average: 0.00, 0.01, 0.05
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.7 us, 0.3 sy, 0.0 ni, 99.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 509428 total, 483252 used, 26176 free, 28132 buffers
KiB Swap: 2333960 total, 0 used, 2333960 free, 373176 cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM   TIME+  COMMAND
 2175 root        20   0 67220  18m 7668  S   0.3   3.7   2:15.03 Xorg
 2332 knoppix     20   0 66516  9492 7692  S   0.3   1.9   0:00.77 metacity
 2350 knoppix     20   0 53356  13m 8904  S   0.3   2.7   0:06.30 lxpanel
 2445 knoppix     20   0 56420  9296 7320  S   0.3   1.8   0:12.01 x-terminal-emul
  1 root         20   0  2364   660  564  S   0.0   0.1   0:00.87 init
  2 root         20   0     0     0     0  S   0.0   0.0   0:00.00 kthreadd
  3 root         20   0     0     0     0  S   0.0   0.0   0:00.37 ksoftirqd/0
  5 root          0 -20     0     0     0  S   0.0   0.0   0:00.00 kworker/0:0H
  6 root         20   0     0     0     0  S   0.0   0.0   0:00.00 kworker/u:0
  7 root          0 -20     0     0     0  S   0.0   0.0   0:00.00 kworker/u:0H
  8 root         rt    0     0     0     0  S   0.0   0.0   0:00.00 migration/0
  9 root         20   0     0     0     0  S   0.0   0.0   0:00.62 rcu_preempt
 10 root         20   0     0     0     0  S   0.0   0.0   0:00.00 rcu_bh
 11 root         20   0     0     0     0  S   0.0   0.0   0:00.00 rcu_sched
 12 root          0 -20     0     0     0  S   0.0   0.0   0:00.00 khelper
 13 root          0 -20     0     0     0  S   0.0   0.0   0:00.00 netns
 433 root         20   0     0     0     0  S   0.0   0.0   0:00.00 bdi-default
```

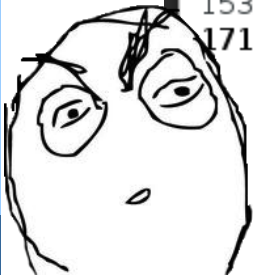




¿Init?

```
oscreader@OSC: ~/Desktop/Labs/4
File Edit View Search Terminal Help
top - 12:03:12 up 36 min, 2 users, load average: 0.13, 0.09, 0.18
Tasks: 133 total, 1 running, 132 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.3 us, 1.0 sy, 0.0 ni, 96.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 771744 total, 708376 used, 63368 free, 94396 buffers
KiB Swap: 392188 total, 96704 used, 295484 free. 262632 cached Mem

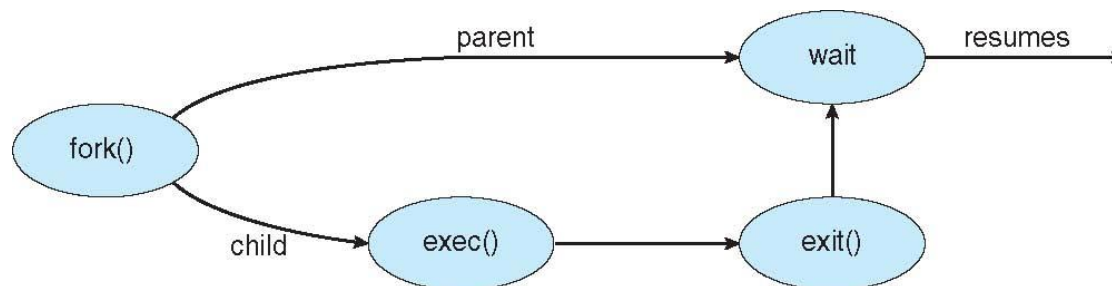
  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 1061 oscread+  20   0 427528 127516 45388 S   2.7  16.5   2:34.22 gnome-shell
   507 root      20   0 111432  25712  6052 S   0.7   3.3   0:39.95 Xorg
 1536 oscread+  20   0  65408  24196 19748 S   0.3   3.1   0:02.04 gnome-termi+
 1716 oscread+  20   0   5080   2552  2188 R   0.3   0.3   0:00.11 top
    1 root      20   0   23956   2660  1724 S   0.0   0.3   0:01.10 systemd
    2 root      20   0         0         0      0 S   0.0   0.0   0:00.00 kthreadd
    3 root      20   0         0         0      0 S   0.0   0.0   0:00.36 ksoftirqd/0
    5 root       0 -20         0         0      0 S   0.0   0.0   0:00.00 kworker/0:0H
    6 root      20   0         0         0      0 S   0.0   0.0   0:00.23 kworker/u2:0
    7 root      20   0         0         0      0 S   0.0   0.0   0:00.67 rcu_sched
    8 root      20   0         0         0      0 S   0.0   0.0   0:00.00 rcu_bh
    9 root      rt    0         0         0      0 S   0.0   0.0   0:00.00 migration/0
   10 root      rt    0         0         0      0 S   0.0   0.0   0:00.02 watchdog/0
   11 root       0 -20         0         0      0 S   0.0   0.0   0:00.00 khelper
   12 root      20   0         0         0      0 S   0.0   0.0   0:00.00 kdevtmpfs
   13 root       0 -20         0         0      0 S   0.0   0.0   0:00.00 netns
   14 root      20   0         0         0      0 S   0.0   0.0   0:00.00 khungtaskd
```





Creación de procesos

- Cuando un proceso crea a otro, el proceso padre puede continuar ejecutándose concurrentemente o esperar a su(s) hijo(s).
 - O ambos. Es posible que el padre ejecute ciertas acciones y luego se detenga a esperar (a uno o más de sus hijos).
- Ejecutar un programa requerirá, además de crear un proceso nuevo, cargar el código y la *data* del programa en cuestión.
 - En sistemas UNIX se usa la llamada `exec()` que reemplaza al programa actual.
 - ▶ La ejecución de programas conlleva crear un proceso (con `fork()`) y cargarle encima otro programa (con `exec()`).



- En Windows, la creación de un proceso nuevo requiere que se indique el programa que dicho proceso ejecutará.





Ejemplos de creación de procesos

Linux (POSIX)

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

Windows (Windows API)

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }

    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





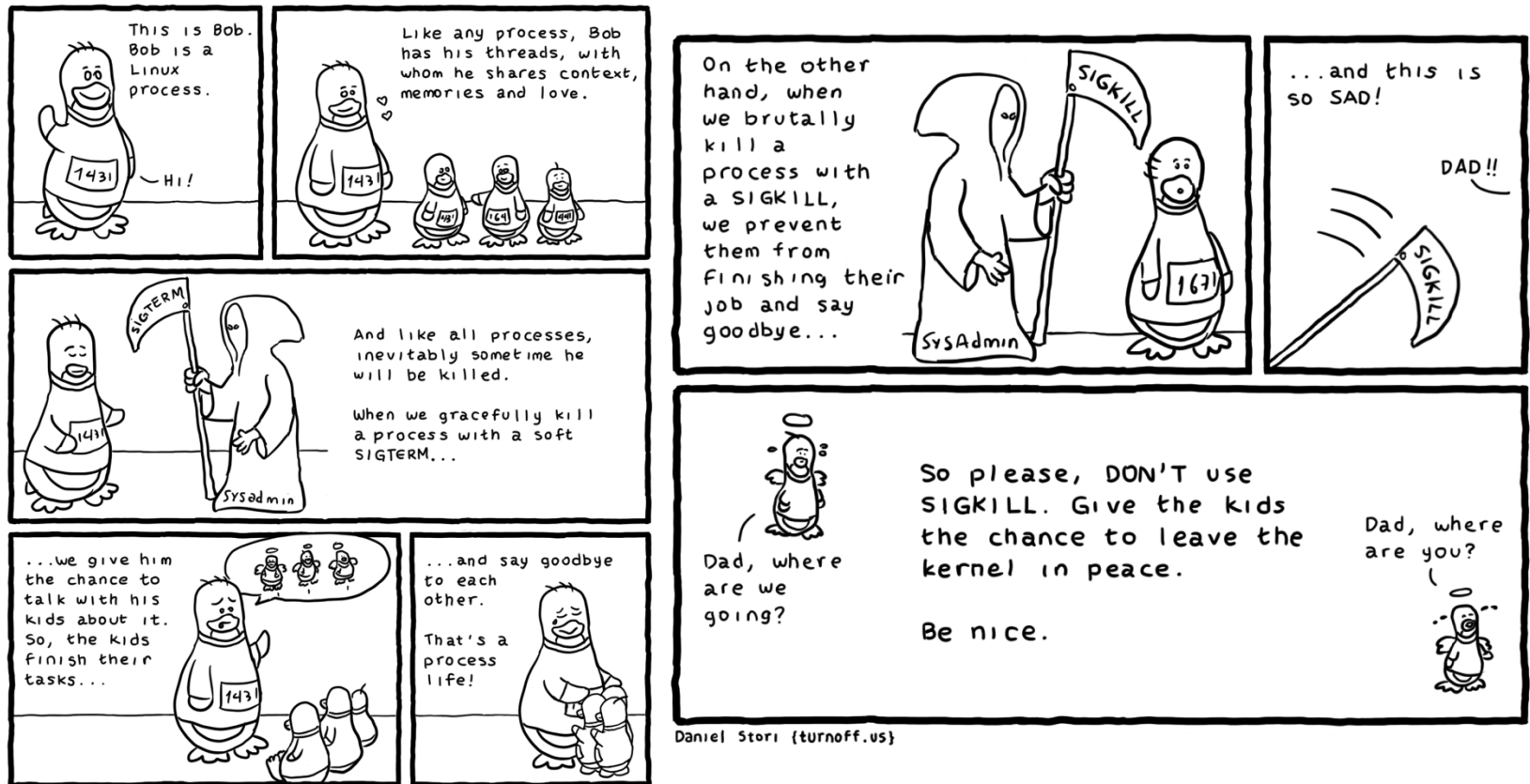
Terminación de procesos

- Un proceso puede terminar su ejecución voluntaria o involuntariamente.
 - El OS provee una *syscall* (en sistemas UNIX, `exit()`) para terminación voluntaria.
 - La terminación involuntaria se da caso de error fatal (e.g., operaciones no permitidas) o mediante otro proceso (que requerirá ciertos privilegios).
- Un proceso padre puede matar a sus hijos (`abort()`) o esperar a que terminen (`wait()`).
 - Es posible que un proceso hijo termine antes de que el padre llegue a esperarlo. En este caso, el hijo queda **zombie**.
- Las posibilidades cuando un proceso padre termina son:
 - Abandonar a los hijos (quedan **huérfanos**).
 - ▶ Los huérfanos suelen ser adoptados por `init`.
 - Llevarse a los hijos a la tumba (**terminación en cascada**).





Terminación de procesos



- La terminación de procesos involucra operaciones administrativas.
 - Los recursos son desalojados por el OS.
 - Los procesos pueden llevar a cabo un *clean-up*.
 - E.g., ejecución de destructores.
- En UNIX se usan **señales** para interrumpir y terminar procesos.





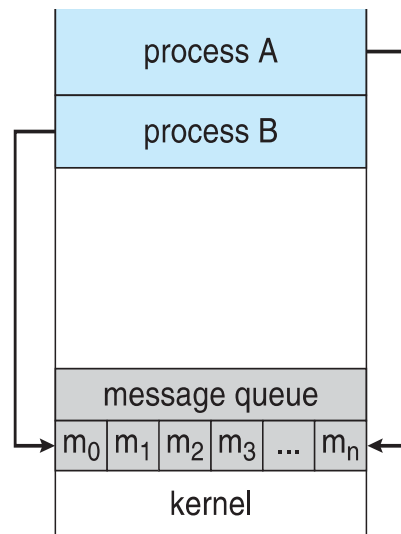
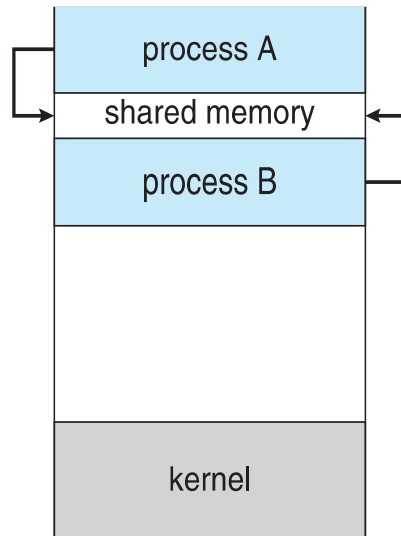
Comunicación entre procesos

- ¿Cuáles son las dos principales formas de comunicación entre procesos? ¿Cómo podemos explicarlas con medios de comunicación externos al entorno computacional?
- ¿Qué implican los tipos de comunicación síncrona y asíncrona? ¿Qué formas de comunicación cotidiana podemos citar de cada categoría?





Comunicación entre procesos



- Motivación para comunicación entre procesos (**IPC**, por *interprocess communication*):
 - Para compartir información y recursos, hay que coordinarse.
 - Para modularizar un sistema y reutilizar código, procesos pueden requerir servicios entre sí.
 - Para “encadenar” tareas y usar el *output* de un proceso como *input* de otro.
 - Para sincronizar ejecución de procesos y establecer un orden entre tareas.
- La comunicación entre procesos puede ocurrir de dos maneras:
 - **Memoria compartida**: procesos tienen acceso a un mismo espacio de memoria en el que pueden leer y escribir.
 - **Paso de mensajes**: procesos envían información a otros siguiendo un protocolo.





Memoria compartida

- Un proceso solicita un espacio de **memoria compartida** que es adherido al espacio de memoria de los procesos en comunicación.
 - El sistema operativo lo provee y da un paso atrás. Los procesos están encargados de lo que pase en ese espacio.

OS



- Suele ser más rápido que paso de mensajes porque requiere menos intervención del sistema operativo.
 - En general, no requiere un protocolo, ni preparación (formato, cifrado, etc.) ni manejo especial (decodificación, descompresión, etc.) de la información.
- Suele ser más problemático que paso de mensajes porque:
 - Es más difícil de implementar, especialmente en sistemas distribuidos.
 - Nos enfrenta a conflictos de acceso y sincronización.





Paso de mensajes

- El paso de mensajes prescinde de una región de memoria compartida en favor de un **enlace de comunicación** (*communication link*).
- El modelo de paso de mensajes emplea al menos dos operaciones básicas:
 - **send(message)**
 - **receive(message)**
- La implementación del *link* y las operaciones de envío y recepción determina la forma en la que la comunicación se lleva a cabo.
 - Comunicación directa o indirecta.
 - Comunicación síncrona o asíncrona.
 - *Buffering* implícito o explícito.





Comunicación directa e indirecta

□ Directa:

- **Simétrica** implica que los procesos especifican a quién envían y de quién reciben.
- **Asimétrica** requiere únicamente que el emisor especifique el destino de su mensaje.
- Los procesos sólo necesitan conocer la identidad del otro. Se establece exactamente un *link* entre exactamente dos procesos.

□ Indirecta implica el uso de un **buzón**. El *link* de comunicación entre dos procesos se establece por medio del buzón.

- El *link* se establece entre pares de procesos que comparten el buzón.
- Un buzón puede facilitar la comunicación entre más de dos procesos.
- Dos procesos se pueden comunicar por medio de más de un buzón.
- ¿Qué pasa si tres procesos comparten un buzón y dos procesos intentan recibir a la vez un mismo mensaje dejado por el tercer proceso?





Comunicación síncrona y asíncrona

- La **sincronía**, en este contexto, se refiere al uso de bloqueos durante la comunicación:
 - **Envío/recepción bloqueadora**: el proceso emisor/receptor se bloquea hasta que el mensaje enviado/esperado sea recibido.
 - **Envío/recepción no-bloqueadora**: el emisor continúa trabajando luego de enviar, y el receptor recibe *null* si no hay nada que recibir.
- Bloqueador = síncrono; no-bloqueador = asíncrono.
- No necesariamente se empuñan las tecnologías en pareja. Se pueden hacer combinaciones.
 - Tener tanto `send` como `receive` bloqueadoras presenta un **rendezvous** entre emisor y receptor.





¿*Bufferear* o no *bufferear*?

- El modelo de paso de mensajes puede hacer uso de **colas de mensajes** (*buffers*) para permitir diferentes grados de asincronía.
- Implementaciones comunes de las colas suelen mantener copias de los mensajes o referencias a los mismos.
- Los casos son tres:
 - Sin *buffering*: cola de capacidad cero. Obliga al emisor a bloquearse hasta que lo enviado sea recibido.
 - *Buffering* automático (finito): la cola tiene capacidad limitada. El emisor se bloquea al enviar un mensaje cuando la cola destino está llena.
 - ▶ Se desbloquea cuando se libera espacio en la cola.
 - *Buffering* automático (ilimitado): el emisor nunca se bloquea porque siempre hay espacio en la cola.





Problema Productor-Consumidor

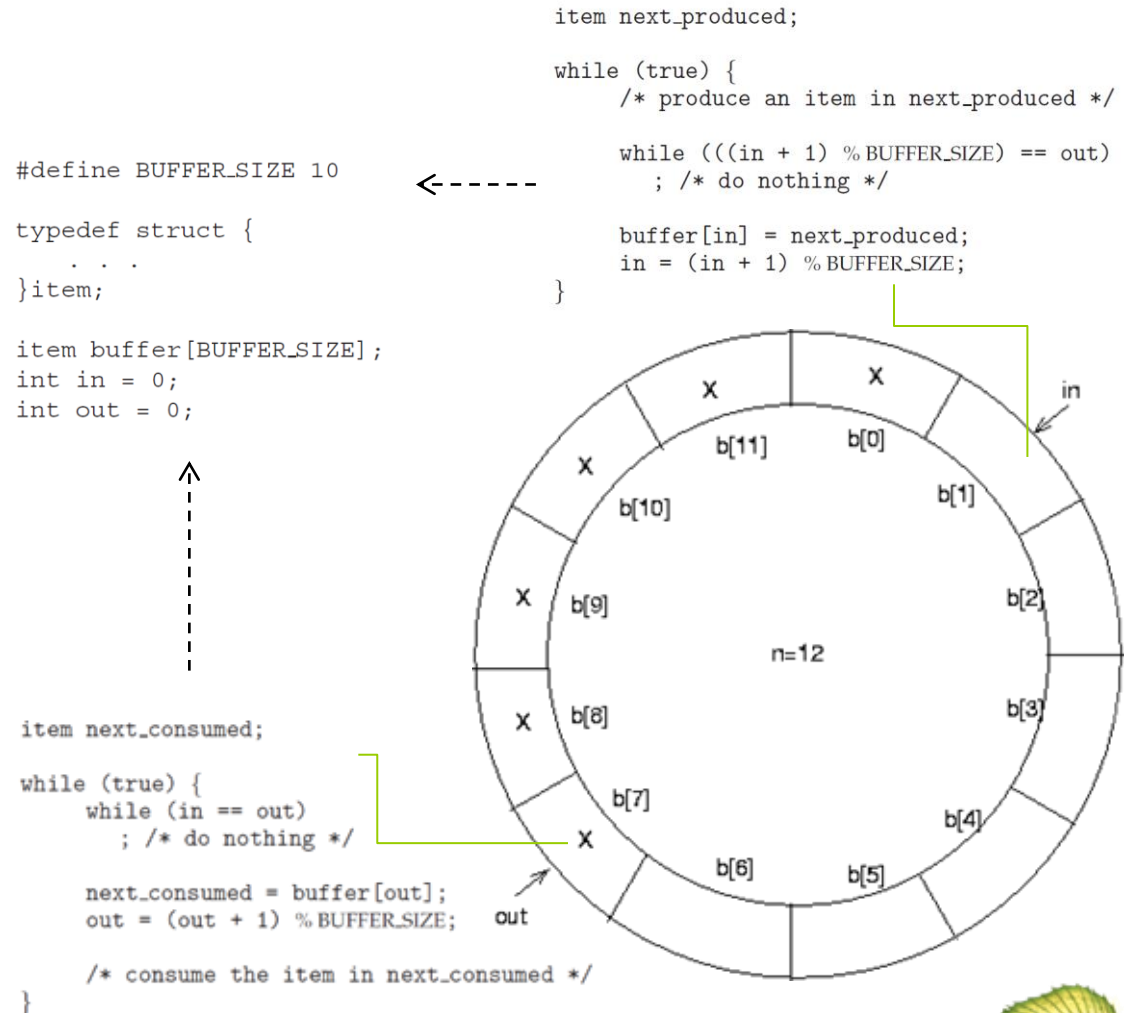
- ¿A qué se refiere el problema de productor-consumidor? ¿Qué ocurrencias de este problema podemos mencionar en nuestra vida cotidiana?
- ¿Qué escenarios computacionales o informáticos podemos asociar con el problema de productor-consumidor?
- ¿Cómo se pueden aplicar los conceptos de *buffering* y llamadas bloqueadoras en la solución al problema productor-consumidor?





Problema Productor-Consumidor

- Paradigma de cooperación; un proceso toma el rol de **productor** de información, otro el de su **consumidor**.
- Una solución emplea un *buffer* (típicamente, finito) en una región de memoria compartida.
 - Los procesos sincronizan su producción y consumo.
 - Con un *buffer* ilimitado sólo el consumidor podría tener que esperar.





Problema Productor-Consumidor

- En el modelo de paso de mensajes, la solución al problema se hace trivial mediante un *rendezvous*.

```
message next_produced;  
  
while (true) {  
    /* produce an item in next_produced */  
  
    send(next_produced);  
}  
  
message next_consumed;  
  
while (true) {  
    receive(next_consumed);  
  
    /* consume the item in next_consumed */  
}
```





Ejemplos de sistemas de IPC- POSIX

- En Linux, un proceso crea un segmento de memoria compartida con la siguiente llamada a sistema:

```
shm_fd = shm_open(name, O_CREAT | O_RDWR,  
0666) ;
```

- También usado para acceder a un segmento compartido existente.
- Crea un espacio de memoria compartida, que puede verse como un archivo en `/dev/shm`.
- El archivo se crea con tamaño 0 por defecto. Para cambiar el tamaño de un archivo (normal o memoria compartida) se usa:

```
ftruncate(shm_fd, 4096) ;
```

- Esto le asigna un tamaño de 4096B.
- Se *mappea* la región de memoria compartida a una porción del espacio de memoria del proceso con la siguiente llamada:

```
void *ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE,  
MAP_SHARED, shm_fd, 0) ;
```

- Esto es lo que provee al proceso de acceso a la memoria compartida.





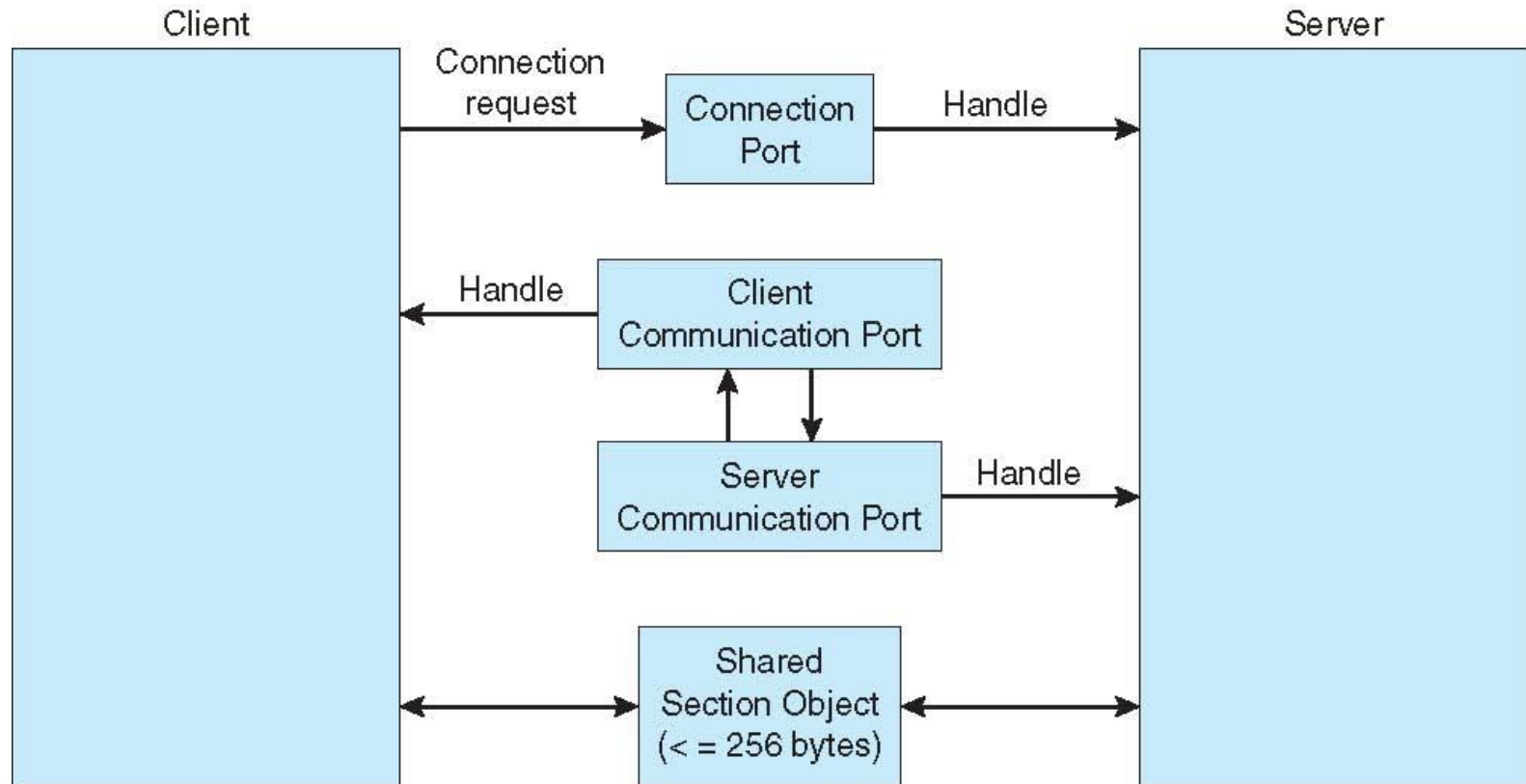
Ejemplos de sistemas de IPC- POSIX

- Un espacio de memoria compartida es liberado con
 - `shm_unlink(name)`
 - Elimina el nombre del objeto de memoria compartida, de modo que `shm_open` ya no funcione con dicho nombre.
 - Espera a que todos los procesos eliminen su mapeo al espacio de memoria compartida antes de destruir dicho espacio.
- `munmap()` elimina el espacio referido por sus parámetros (dirección y tamaño) del espacio de direcciones del proceso.
 - Es una forma voluntaria de eliminar un mapeo, pero se hace automáticamente cuando el proceso termina.
 - No es lo mismo que cerrar el *file descriptor* para un objeto de memoria compartida.





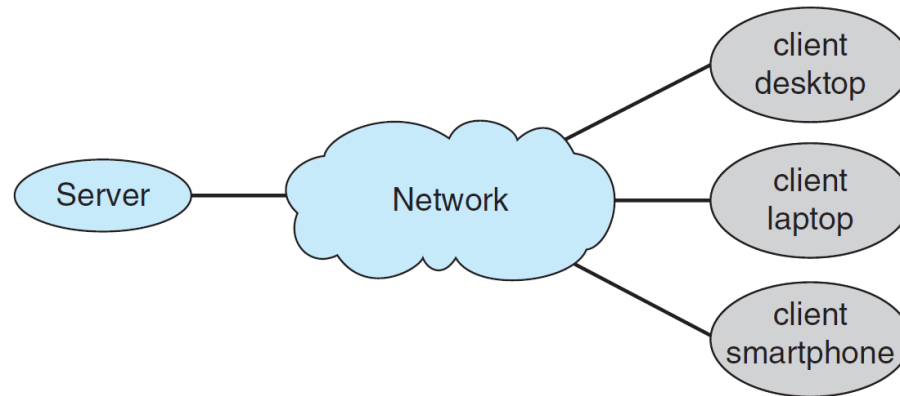
Ejemplos de sistemas IPC - Windows





Sistemas Cliente-Servidor

- El problema de productor-consumidor plantea una situación de comunicación en una vía.
 - Es posible “encadenar” la comunicación entre productores y consumidores, haciendo que la salida de uno sea entrada del otro.
- Los **sistemas cliente-servidor** están pensados para comunicación en dos vías.
 - Un servidor realiza una tarea especializada. Los clientes envían solicitudes al servidor y el servidor responde con el resultado.





Sockets

- ❑ ¿Qué tipo de comunicación entre procesos proveen los *sockets*?
- ❑ ¿Cuántos *sockets* están involucrados en la comunicación entre dos procesos? ¿Qué hay del escenario en el que un proceso es servidor y más de un cliente se conecta a él?
- ❑ ¿Cuál es la importancia del protocolo de comunicación?





Sockets

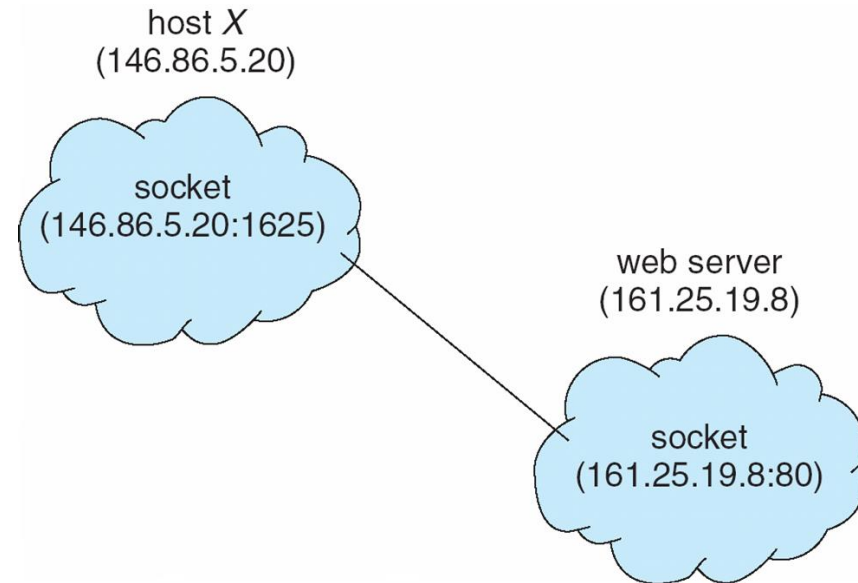
- Un **socket** es un punto (*endpoint*) de comunicación para un sistema.
 - Frecuentemente usados para comunicación entre computadoras a través de una red (mediante el protocolo TCP).
 - ▶ Se identifican con una **dirección IP** y un **puerto de software** (e.g., **161.25.19.8:1625**).
 - ▶ La dirección 127.0.0.1 es conocida como el **loopback**: una autoreferencia.
 - ▶ Los **puertos bien conocidos** (*well known ports*) tienen números del 1 al 1024. Suelen ser usados para tipos específicos de comunicación.
 - También usables para comunicación local (IPC).
- Los **sockets** son un recurso de conexión provisto por el sistema operativo.
 - Un proceso solicita la creación de un **socket** para escuchar cierto puerto en el sistema.
 - El sistema operativo crea el **socket** y provee una referencia al proceso solicitante.





Sockets

- La comunicación se establece entre dos *sockets*.
- Las conexiones son únicas.
 - Involucran una pareja única de *sockets*.
 - Recordemos que los *sockets* se identifican por dirección:puerto.
- Detrás de un *socket* está la ***protocol stack***.
 - Los *sockets* son los medios de acceso a la *protocol stack*.
 - La *protocol stack* implementa, en el OS, los protocolos de comunicación soportados.





Sockets: servidor de chat en Py

Deprecada

```
import asyncio
import socket
```

```
clients = {}
```

```
class MainServerSocket(asyncio.dispatcher):
```

```
    def __init__(self):
```

```
        asyncio.dispatcher.__init__(self)
```

```
        self.create_socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
        self.bind(('0.0.0.0', 21567))
```

```
        self.listen(5)
```

```
    def handle_connection(self):
```

```
        newSocket, address = self.accept()
```

```
        clientSocket = newSocket
```

```
        clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
        clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
```

```
        clientSocket.setblocking(0)
        clientSocket.start(asyncio.dispatcher.with_send):
```

Socket binding es la asignación de dirección puerto al socket creado. Se suele decir que se le asigna "nombre" al socket.

```
        message = receivedData[(separation+1):]
```

```
        every = clients.items()
```

```
        if (destination == "0"):
```

```
            for addr, one in every:
```

```
                one.send((str(one.getpeername()[0]) + ': ' + message).encode('utf-8'))
```

```
        else:
```

```
            for addr, one in every:
```

```
                if (str(addr[0]) == destination):
```

```
                    one.send((str(one.getpeername()[0]) + " says: " + message).encode('utf-8'))
```

```
        else: self.close()
```

```
    def handle_close(self):
```

```
        print ("Disconnected from", self.addr)
```

```
        one = self.addr
```

```
        del clients[one]
```

```
MainServerSocket()
```

```
asyncio.loop().run_forever() #Polling loop que consulta diferentes canales (sockets) abiertos mediante instancia
```

asyncio.dispatcher es *Wrapper Class* para *sockets*. Esta clase se crea para establecer un *socket* en el servidor encargado de aceptar nuevas conexiones.

Al instanciarse esta clase, lo ultimo que hace es quedarse pendiente de solicitudes de conexión.

La creación del *socket*. Los parámetros especifican el formato esperado de direcciones que entiende el *socket* (en este caso, IPv4) y el tipo de *socket*, i.e., el protocolo de comunicación (en este caso, TCP).





Sockets: servidor de chat en Py

```
import asyncio
import socket
```

```
clients = {}
```

Aceptar una conexión provee un nuevo *socket* y la dirección del otro extremo en la conexión.

```
.dispatcher):
```

```
__init__(self)
```

```
server = socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
self.bind((IP, 567))
```

```
self.listen(5)
```

```
def handle_accept(self,
```

```
newSocket, address = se
```

```
clients[address] = newS
```

```
print ("Connected from"
```

```
SecondaryServerSocket(newSocket,
```

Se instancia la clase *SecondaryServerSocket* con el nuevo *socket*, creado al aceptar la conexión.

```
class SecondaryServerSocket(asyncio.dispatcher_with_send):
```

```
def handle_read(self):
```

```
receivedData = self.recv(8192)
```

```
if receivedData:
```

```
    receivedData = receivedData.decode('utf-8')
```

```
    separation = receivedData.index("/")
```

```
    destination = receivedData[:separation]
```

```
    message = receivedData[(separation+1):]
```

```
    every = clients.items()
```

```
    if (destination == "0"):
```

```
        for addr, one in every:
```

```
            one.send((str(one.getpeername()[0]) + ': ' + message).encode('utf-8'))
```

```
        for addr, one in every:
```

```
            if (str(addr[0]) == destination):
```

```
                one.send((str(one.getpeername()[0]) + " says: " + message).encode('utf-8'))
```

```
            close()
```

```
        self:
```

```
print ("Disconnected from", self.addr)
```

```
one = self.addr
```

```
del clients[one]
```

```
MainServerSocket()
```

```
asyncio.loop() #Polling loop que consulta diferentes canales (sockets) abiertos mediante instancia
```





Sockets: cliente de chat en Py

```
from tkinter import *
from socket import *
import _thread

HOST = '10.97.115.35'
PORT = 21567
BUFSIZE = 1024
ADDR = (HOST, PORT)

tcpCliSock = socket(AF_INET, SOCK_STREAM)
tcpCliSock.connect(ADDR)

class Application(Frame):
    def __init__(self, master):
        Frame.__init__(self, master)
        self.grid()
        self.create_widgets()
        self.socket()

    def callback(self, event):
        message = str(self.entry_field.get())
        tcpCliSock.send(message.encode('utf-8'))

    def create_widgets(self):
        self.messaging_field = Text(self, width = 110, height = 20, wrap = WORD)
        self.messaging_field.grid(row = 0, column = 0, columnspan = 2, sticky = W)

        self.entry_field = Entry(self, width = 92)
        self.entry_field.grid(row = 1, column = 0, sticky = W)
        self.entry_field.bind('<Return>', self.callback)

    def add(self, data):
        self.messaging_field.insert(END, data)

    def socket(self):
        def loop0():
            while 1:
                data = tcpCliSock.recv(BUFSIZE)
                if data: self.add(data)
        _thread.start_new_thread(loop0, ())

root = Tk()
root.title("Chat client")
root.geometry("550x260")

app = Application(root)

root.mainloop()
```





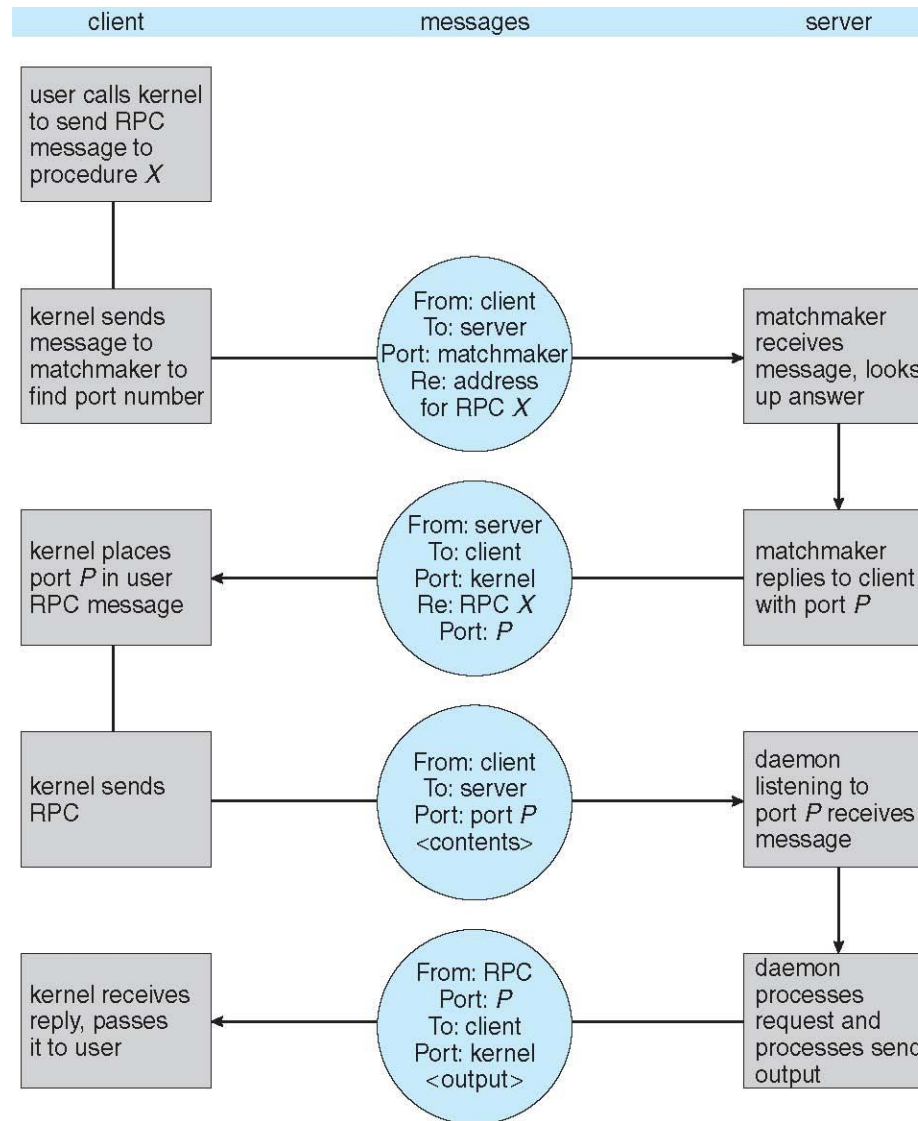
Remote Procedure Calls (RPC)

- La gracia es que el cliente pueda usar servicios como si fueran métodos locales.
 - Abstrae los detalles de la comunicación.
 - Es una forma de paso de mensajes.
- Ventajas adicionales:
 - Facilita la documentación de operaciones con interacción remota (compárese el caso con el uso de *sockets* directamente).
 - Permite automatizar los procesos de comunicación.
 - Facilita portabilidad del código.
- Problemas potenciales:
 - Representación de la información: los datos pueden tener diferentes representaciones en cliente y servidor.
 - Problemas de red.
 - ¿Cómo conoce el cliente a dónde dirigir sus llamadas en el servidor?
 - Difícil implementación de paso de parámetros por referencia.





Remote Procedure Calls (RPC)





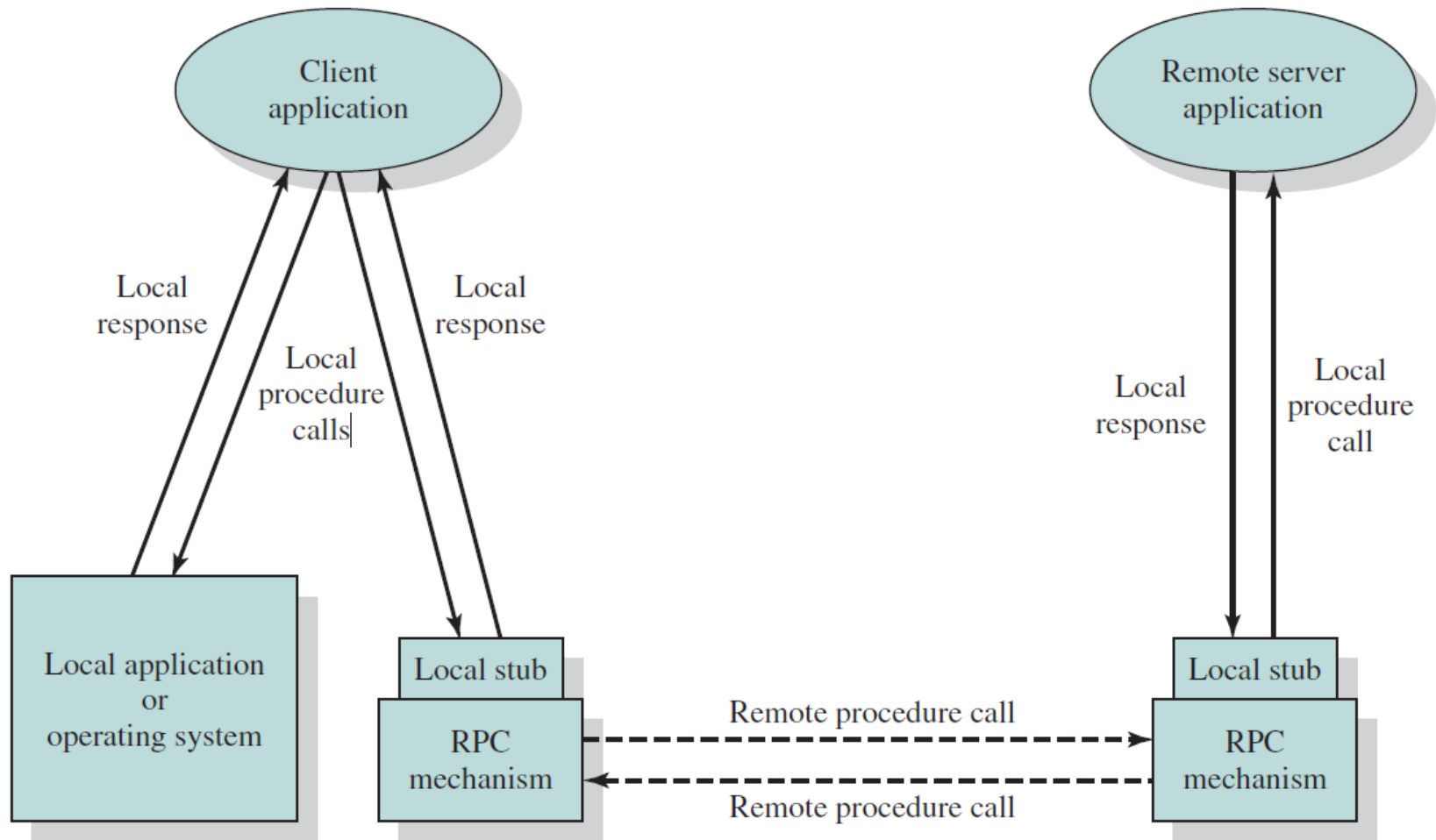
Remote Procedure Calls (RPC)

- La implementación común es mediante *stubs*. Un *stub* en el cliente y uno en el servidor.
 - Los *stubs* son locales, pero se encargan de procesar la llamada para envío y recepción con la contraparte.
- En el servidor se asocian procedimientos con puertos.
 - El *stub* del cliente se encarga de referenciar al puerto correspondiente y con los parámetros correspondientes.
 - Los parámetros de la RPC realizada deben transformarse para envío a través de una red (por ejemplo, convertir una imagen a un *stream* de datos).
 - ▶ A esto se le llama *marshalling* de parámetros.
- Cuestiones a considerar:
 - Sincronía: sacrifica paralelismo.
 - Persistencia de la conexión: conveniente para procesos que efectúan muchas llamadas.
 - Orientación a objetos.





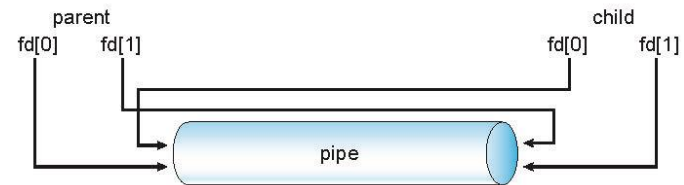
Remote Procedure Calls (RPC)





Pipes

- “Tubos” de comunicación entre dos procesos, proporcionados por el sistema operativo.
- Los hay **ordinarios** y **nombrados**.
 - Ordinarios: unidireccionales. Para comunicación entre procesos padre e hijo (por ende, entre procesos en la misma computadora).
 - ▶ En Windows son llamados **anónimos**.



- Nombrados: bidireccionales, persistentes. Para comunicación entre procesos no necesariamente relacionados.
 - ▶ En sistemas UNIX, son un tipo especial de archivo creado con la function **mkfifo**.
 - ▶ Con el archivo se lee y escribe como con cualquier otro, pero la apertura del archivo puede ser bloqueadora.





Fuentes

- ❑ https://www.tutorialspoint.com/operating_system/os_types.htm
- ❑ http://www.faqs.org/docs/bashman/bashref_78.html#SEC85
- ❑ https://www.ibm.com/support/knowledgecenter/zosbasics/com.ibm.zos.zmainframe/zconc_batchproc.htm
- ❑ <https://www.microsoftpressstore.com/articles/article.aspx?p=2233328>
- ❑ <http://www.pcworld.com/article/2841873/meet-systemd-the-controversial-project-taking-over-a-linux-distro-near-you.html>
- ❑ https://www.webopedia.com/TERM/P/protocol_stack.html
- ❑ <http://man7.org/linux/man-pages/man2/bind.2.html>
- ❑ <https://ired.team/miscellaneous-reversing-forensics/windows-kernel/manipulating-activeprocesslinks-to-unlink-processes-in-userland>
- ❑ <https://linux.die.net/man/7/fifo>
- ❑ <https://www.tecmint.com/systemd-replaces-init-in-linux/>





Fuentes

- ❑ van der Linden, P. (1994). *Expert C Programming: Deep C Secrets*. SunSoft Press.
- ❑ Anderson, T., & Dahlin, M. (2012). *Operating Systems: Principle and Practice*.
- ❑ Intel. (2018). *Intel (R) 64 and IA-32 Architectures: Software Developer's Manual (Combined Volumes)*.
- ❑ Silberschatz, A., Galvin, P., & Gagne, G. (2013). *Operating System Concepts*. John Wiley & Sons, Inc.
- ❑ Tanenbaum, A. (2009). *Modern Operating Systems*. Pearson Education, Inc.
- ❑ Tanenbaum, A., & Woodhull, A. (2006). *Operating Systems: Design and Implementation*. Prentice Hall.
- ❑ Stallings, W. (2012). *Operating Systems: Internals and Design Principles*. New Jersey: Prentice Hall.
- ❑ Russinovich, M. E., Solomon, D. A., & Ionescu, A. (2012). *Windows Internals, Part1*. Pearson Education

